On the other hand, a singleton can be considered an anti-pattern because it can become a single point of failure or a bottleneck in the application. This is because if the singleton has a critical responsibility for the system, the application breaks when it fails. Whether you use it or not, every decision is a tradeoff.

When an Akka singleton is respawned in another node, there is downtime in the relocation. Another problem is that singletons live in the oldest node. So if an application has multiple singletons, this node may be overloaded.

To create a singleton, you need a singleton factory: the behavior you want to use as a singleton and a singleton wrapper that wraps that behavior. Your behavior may look something like this:

```
object MyBehavior {
  def apply(): Behavior[String] = Behaviors.Ignore
}
```

If you have an `ActorSystem`, you can create the singleton factory as follows:

```
val singletonFactory = ClusterSingleton(system)
```

Then wrap the behavior in a `SingletonActor` and set up the supervision strategy to `restart`:

```
Val wrappedBehavior = SingletonActor(
                          Behaviors
                      .supervise(MyBehavior())
                      .onFailure(SupervisorStrategy.restart),
                          "myShardingCoordinator"))
```

Now you can create the singleton: